

Exercicio da Aula 07 – Exercicio de Comandos Aggregate

Baseado nos exemplos executados e na aula sobre aggregate, elaborar alguns comandos do MongoDB

DATABASE BIBLIOTECA

[Questão 01] Criar um comando Aggregate associando as collections LIVROS e EMPRESTIMO listando todos os EMPRESTIMOS

```
db.Emprestimo.aggregate([
  {
    $lookup: {
      from: "livros",          // nome da outra coleção
      localField: "_id_livro", // campo em Emprestimo
      foreignField: "_id",     // campo em Livros
      as: "livro"              // nome do campo que será adicionado
    }
  }
])
```

[Questão 02] Criar um comando Aggregate associando as collections LIVROS e EMPRESTIMO listando o EMPRESTIMOS com _id = 1

```
db.Emprestimo.aggregate([
  { $match: { _id: 1 } }, // Filtra o empréstimo desejado
  { $unwind: "$emprestimos" }, // Desagrega o array "emprestimos"
  {
    $lookup: {
      from: "Livros",
      localField: "emprestimos._id_livro",
      foreignField: "_id",
      as: "livro"
    }
  },
  {
    $project: {
      _id_livro: "$emprestimos._id_livro",
      // Correção: "Titulo" → "titulo" (se o campo estiver em minúsculo)
      nome_livro: { $arrayElemAt: ["$livro.titulo", 0] } // [1]
    }
  }
]);
```

[Questão 03] Criar um comando Aggregate associando as collections LIVROS e EMPRESTIMO listando o LIVROS com _id = 1

```
db.Emprestimo.aggregate([
{
  // Garante que o campo 'emprestimos' existe, senão devolve array vazio
  $project: {
    emprestimos: {
      $cond: {
        if: { $isArray: "$emprestimos" },
        then: "$emprestimos",
        else: []
      }
    }
  }
},
{
  // Adiciona os índices ao array
  $project: {
    emprestimosComIndice: {
      $map: {
        input: { $range: [0, { $size: "$emprestimos" }] },
        as: "idx",
        in: {
          index: "$$idx",
          item: { $arrayElemAt: ["$emprestimos", "$$idx"] }
        }
      }
    }
  }
},
{
  // Filtra os itens que têm _id_livro = 1
  $project: {
    indices: {
      $filter: {
        input: "$emprestimosComIndice",
        as: "e",
        cond: { $eq: ["$$e.item._id_livro", 1] }
      }
    }
  }
},
{
  // Só mantém os documentos que têm pelo menos um match
  $match: {
    "indices.0": { $exists: true }
  }
},
{
  // Retorna somente o _id e os índices onde o livro com id 1 aparece
  $project: {
    _id: 1,
    indices: {
```

```
$map: {  
  input: "$indices",  
  as: "e",  
  in: "$$e.index"  
}  
}  
}  
}  
})
```

[Questão 04] Criar um comando Aggregate associando as collections EMPRESTIMOS e USUARIO listando todos os EMPRESTIMOS

```
db.Emprestimo.aggregate([  
  {  
    $lookup: {  
      from: "Usuario",  
      localField: "_id",  
      foreignField: "_id",  
      as: "usuario"  
    }  
  },  
  {  
    $unwind: "$usuario"  
  },  
  {  
    $project: {  
      _id: 0,  
      emprestimo: "$$ROOT",  
      usuario: 1  
    }  
  }  
])
```

[Questão 05] Criar um comando Aggregate associando as collections EMPRESTIMOS e USUARIO listando todos os EMPRESTIMOS com _id = 1

```
db.Emprestimo.aggregate([  
  {  
    // Etapa para projetar somente o segundo empréstimo (índice 1) de cada documento  
    $project: {  
      _id: 1,  
      emprestimo: { $arrayElemAt: ["$emprestimos", 1] }  
    }  
  },  
  {  
    // Etapa de junção com a collection usuario  
    $lookup: {  
      from: "Usuario",  
      localField: "_id",  
      foreignField: "_id",  
      as: "usuario"  
    }  
  },  
  {
```

```
// Etapa para transformar o array "usuario" em objeto
$unwind: "$usuario"
},
{
// Etapa final: projetar os campos desejados
$project: {
  _id: 1,
  usuario: {
    nome: "$usuario.nome",
    cpf: "$usuario.cpf",
    telefone: "$usuario.telefone",
    cidade: "$usuario.cidade",
    uf: "$usuario.uf"
  },
  emprestimo: {
    id_livro: "$emprestimo._id_livro",
    assunto: "$emprestimo.Assunto",
    data_emprestimo: "$emprestimo.Data_Emprestimo",
    data_devolucao: "$emprestimo.Data_Devolucao",
    multa: "$emprestimo.Multa"
  }
}
}
})
```

[Questão 06] Criar um comando Aggregate associando as collections EMPRESTIMOS e USUARIO listando todos os USUARIOS com _id = 1

```
db.Emprestimo.aggregate([
// Etapa de correspondência para encontrar os empréstimos com _id igual a 1
{ $match: { _id: 1 } },

// Etapa de junção com a coleção de Usuario para buscar detalhes do usuário
{
  $lookup: {
    from: "Usuario",
    localField: "_id",
    foreignField: "_id",
    as: "usuario"
  }
}
```

```
}  
},  
  
// Etapa de desenrolar para desagregar o array de usuários (caso necessário)  
{ $unwind: "$usuario" },  
  
// Etapa de projeto para formatar a saída conforme necessário  
{  
  $project: {  
    _id: 0,  
    emprestimo: "$$ROOT",  
    usuario: 1  
  }  
}  
})
```

DATABASE LOJA_DEPARTAMENTOS

[Questão 07] Criar um comando Aggregate associando as collections VENDAS e PRODUTOS

```
db.Vendas.aggregate([  
  {  
    $lookup: {  
      from: "Produtos",  
      localField: "_id",  
      foreignField: "_id",  
      as: "produto"  
    }  
  },  
  {  
    $unwind: "$produto" // Desnormaliza o array de produtos  
  },  
  {  
    $project: {  
      _id: 0, // Exclui o campo _id  
      venda: "$$ROOT", // Mantém o documento original de Vendas  
      produto: 1 // Inclui os detalhes do produto  
    }  
  }  
])
```

[Questão 08] Criar um comando Aggregate associando as collections VENDAS e PRODUTOS, listando a VENDA com _id = 1

```
db.Vendas.aggregate([
  // Filtra apenas a venda com _id igual a 1
  { $match: { _id: 1 } },

  // Desestrutura o array de vendas
  { $unwind: "$vendas" },

  // Faz o lookup com Produtos usando o campo vendas.Id_produto
  {
    $lookup: {
      from: "Produtos",
      localField: "vendas.Id_produto",
      foreignField: "_id",
      as: "produto"
    }
  },

  // Desestrutura o array retornado pelo lookup
  { $unwind: "$produto" },

  // Projeta os campos desejados
  {
    $project: {
      _id: 0,
      id_venda: "$_id",
      id_produto: "$vendas.Id_produto",
      tipo: "$vendas.Tipo",
      data_compra: "$vendas.Data_compra",
      quantidade: "$vendas.Quantidade",
      valor_total: "$vendas.Valor",
      nota_fiscal: "$vendas.Nota_Fiscal",
      nome_produto: "$produto.Produto",
      fabricante: "$produto.Fabricante",
      preco_catalogo: "$produto.Preco"
    }
  }
])
```

[Questão 09] Criar um comando Aggregate associando as collections VENDAS e PRODUTOS, listando o PRODUTO com _id = 1

```
db.Vendas.aggregate([
  {
    // Garante que o campo 'vendas' é um array
    $match: {
      vendas: { $type: "array" }
    }
  },
  {
    // Mapeia o array de vendas para incluir o índice
    $project: {
      _id: 1,
      indices: {
        $map: {
          input: { $range: [0, { $size: "$vendas" }] },
          as: "i",
          in: {
            $cond: [
              { $eq: [ { $getField: { input: { $arrayElemAt: ["$vendas", "$$i"] }, field: "Id_produto" } }, 1 ] },
              "$$i",
              null
            ]
          }
        }
      }
    }
  },
  {
    // Remove os índices nulos
    $project: {
      _id: 1,
      indices: {
        $filter: {
          input: "$indices",
          as: "idx",
          cond: { $ne: ["$$idx", null] }
        }
      }
    }
  },
  {
    // Filtra apenas os documentos onde o produto com id 1 foi encontrado
    $match: {
      indices: { $ne: [] }
    }
  }
])
```

[Questão 10] Criar um comando Aggregate associando as collections VENDAS e CLIENTE

```
db.Vendas.aggregate([
  {
    $lookup: {
```

```
from: "Cliente",
localField: "_id",    // ID da venda que também é o ID do cliente
foreignField: "_id",  // ID do cliente
as: "cliente"
}
},
{
  $unwind: "$cliente"    // Tira o array de cliente, deixa como objeto
},
{
  $project: {
    _id: 0,              // Não mostrar o _id raiz
    cliente: 1,          // Mostrar o cliente
    vendas: 1            // Mostrar as vendas
  }
}
})
```

[Questão 11] Criar um comando Aggregate associando as collections VENDAS e CLIENTE com o _Id = 1 de CLIENTE

```
db.Cliente.aggregate([
  { $match: { _id: 1 } },
  {
    $lookup: {
      from: "Vendas",
      localField: "_id",
      foreignField: "_id",
      as: "vendas"
    }
  },
  {
    $project: {
      _id: 0,
      cliente: "$$ROOT",
      vendas: 1
    }
  }
])
```


[Questão 12] Criar um comando Aggregate associando as collections VENDAS e CLIENTE com o _id = 1 de VENDAS

```
db.Vendas.aggregate([
  // Etapa de correspondência para encontrar a venda com _id igual a 1
  { $match: { _id: 1 } },

  // Etapa de junção com a coleção de Cliente para buscar detalhes do cliente
  {
    $lookup: {
      from: "Cliente",
      localField: "_id",
      foreignField: "_id",
      as: "cliente"
    }
  },

  // Etapa de desenrolar para desagregar o array de clientes (caso necessário)
  { $unwind: "$cliente" },

  // Etapa de projeto para formatar a saída conforme necessário
  {
    $project: {
      _id: 0,
      venda: "$$ROOT",
      cliente: 1
    }
  }
])
```